



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

QUESTIONS 4 / 8 completed

1-10 T

1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java

Run Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Thread t = new Thread() -> {
4             for (int i = 1; i <= 10; i++) {
5                 System.out.println(i);
6             }
7         };
8         t.start();
9     }
10 }
```

OUTPUT

INPUT

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

QUESTIONS 4 / 8 completed

Counter-T

COUNTER-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java

Run Save

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         Counter counter = new Counter();
4         Thread t1 = new Thread() -> {
5             for (int i = 0; i < 1000; i++) counter.increment();
6         };
7         Thread t2 = new Thread() -> {
8             for (int i = 0; i < 1000; i++) counter.increment();
9         };
10        t1.start();
11        t2.start();
12        t1.join();
13        t2.join();
14        System.out.println("Final Counter Value: " + counter.getCount());
15    }
16 }
17 class Counter {
18     private int count = 0;
19     public synchronized void increment() {
20         count++;
21     }
22 }
```

OUTPUT

INPUT

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

QUESTIONS 4 / 8 completed

M-Thread

M-THREAD
Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR Java Run Save

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         long startTime = System.currentTimeMillis();
4         Thread t1 = new Thread(new Task("Thread-1"));
5         Thread t2 = new Thread(new Task("Thread-2"));
6         Thread t3 = new Thread(new Task("Thread-3"));
7         t1.start();
8         t2.start();
9         t3.start();
10        t1.join();
11        t2.join();
12        t3.join();
13        long endTime = System.currentTimeMillis();
14        System.out.println("\nAll tasks completed!");
15        System.out.println("Total Time Taken: " + (endTime - startTime) + "ms");
16    }
17 }
18 class Task implements Runnable {
19     String name;
20
21     Task(String name) {
```

OUTPUT
Thread-2 started...

INPUT
Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

QUESTIONS 4 / 8 completed

Pro-Consume

PRO-CONSUME
Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR Java Run Save

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 public class Main {
5     public static void main(String[] args) {
6         SharedBuffer buffer = new SharedBuffer(5); // Buffer with max capacity 5
7
8         Thread producer = new Thread(new Producer(buffer), "Producer");
9         Thread consumer = new Thread(new Consumer(buffer), "Consumer");
10
11         producer.start();
12         consumer.start();
13     }
14 }
15
16 class SharedBuffer {
17     private Queue<Integer> queue = new LinkedList<>();
18     private int capacity;
19
20     public SharedBuffer(int capacity) {
21         this.capacity = capacity;
```

OUTPUT
Compilation Error:

INPUT
Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192365010@RD

QUESTIONS 4 / 8 completed

Deadlock

DEADLOCK
Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

JavaFailed to save answer.RunSave

```
1 public class Main {
2     public static void main(String[] args) {
3         Object lock1 = new Object();
4         Object lock2 = new Object();
5
6         Thread t1 = new Thread() -> {
7             synchronized (lock1) {
8                 sleep(100);
9                 synchronized (lock2) {
10                     System.out.println("T1 acquired both locks");
11                 }
12             }, "Thread-1");
13
14         Thread t2 = new Thread() -> {
15             synchronized (lock2) {
16                 sleep(100);
17                 synchronized (lock1) {
18                     System.out.println("T2 acquired both locks");
19                 }
20             }
21         }
22     }
23 }
```

OUTPUT

INPUT

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192365010@RD

QUESTIONS 4 / 8 completed

Wait-Notify

WAIT-NOTIFY
Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

JavaRunSave

```
1 public class Main {
2     public static void main(String[] args) {
3         Message message = new Message();
4
5         Thread sender = new Thread(new Sender(message), "Sender");
6         Thread receiver = new Thread(new Receiver(message), "Receiver");
7
8         receiver.start();
9         sender.start();
10    }
11 }
12
13 class Message {
14     private String content;
15     private boolean available = false;
16
17     // Receiver calls this method
18     public synchronized String read() {
19         // While no message is available, wait
20         while (!available) {
21             try {
22                 wait();
23             } catch (InterruptedException e) {}
24         }
25         return content;
26     }
27
28     // Sender calls this method
29     public synchronized void write(String content) {
30         while (available) {
31             try {
32                 wait();
33             } catch (InterruptedException e) {}
34         }
35         this.content = content;
36         notify();
37     }
38 }
```

OUTPUT

INPUT

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

Pool

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3 import java.util.concurrent.TimeUnit;
4
5 public class Main {
6     public static void main(String[] args) throws InterruptedException {
7         ExecutorService executor = Executors.newFixedThreadPool(3);
8         for (int i = 1; i <= 6; i++) {
9             int taskId = i;
10            executor.execute(new Task(taskId));
11        }
12        executor.shutdown();
13        executor.awaitTermination(60, TimeUnit.SECONDS);
14
15        System.out.println("All tasks completed!");
16    }
17 }
18
19 class Task implements Runnable {
20     private int taskId;
21 }
```

OUTPUT

Task 2 started by pool-1-thread-2
Task 1 started by pool-1-thread-1
Task 3 started by pool-1-thread-3
Task 4 started by pool-1-thread-2
Task 5 started by pool-1-thread-1
Task 6 started by pool-1-thread-3
All tasks completed!

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

Kanigasri M
192565010RD

Join

JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         System.out.println("Main thread started");
4
5         Thread t1 = new Thread(new Task("Thread-1"));
6         Thread t2 = new Thread(new Task("Thread-2"));
7
8         t1.start();
9         t1.join();
10        t2.start();
11        t2.join();
12        System.out.println("Main thread finished after both threads completed");
13    }
14 }
15
16 class Task implements Runnable {
17     String name;
18
19     Task(String name) {
20         this.name = name;
21     }
22 }
```

OUTPUT

Main thread started
Thread-1 started...
Thread-1 finished!

Your INPUT goes here!